

D7032E Home Exam - Helge Lindgren

User's notes:

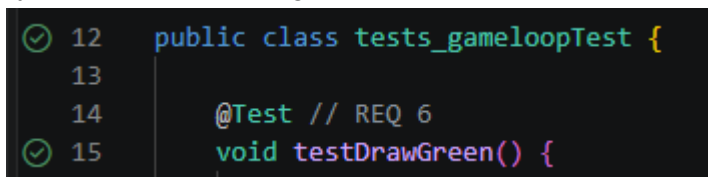
The game can be played through IDEs such as VS code if java extensions have been installed, simply through pressing the run button in the main.java file.

If you wish to launch the game in a terminal, you can compile it with javac. One way of doing this is through visiting each folder containing .java files and running **javac *.java**, which will generate class files for each java file in that directory. The game can then be played by using the command **java src/main/main** from the topmost directory in the project.

This program was tested on jdk 25 and newer, if the code does not want to run as intended, make sure your java version is up to date.

The file loading is implemented in the same way as in the original version, and thus comes with the same caveats regarding file name. If the program encounters errors in trying to load the decks, try using the absolute path instead of a relative path, and then recompile cardDeck.java.

The unit tests can easily be run through the IDE in VS code, simply through pressing the green checkmark icon besides the class or method declaration(may display as other symbols before running the tests)



```
12 public class tests_gameloopTest {
13
14     @Test // REQ 6
15     void testDrawGreen() {
```

The tests use JUnit 6, which may need to be set up on the device for the tests to function. This can be done in VS code according to the following guide:

<https://code.visualstudio.com/docs/java/java-testing>

This home exam was written in parallel to that of Amanda Lorné (Amalor-0), the design of the solution was formulated through discussions with her. All the code in the assignment I am handing in was written by me alone, just as her submission was entirely written by her.

1. Unit testing (2p, max 1 page)

Which requirement(s) (rules and requirements 1 – 15 on previous page) is/are currently not being fulfilled by the code (refer to the requirement number in your answer)? For each of the requirements that are not fulfilled answer:

- If it is possible to test the requirement using JUnit without modifying the existing code, write what the JUnit assert (or appropriate code for testing it) for it would be.
- If it is not possible to test the requirement using JUnit without modifying the existing code, motivate why it is not.

Since most of the code runs in one monolithic block within the **apples2apples** constructors, it is very difficult to isolate any one part of it for testing. Requirements 1-5 all happen as the **apples2apples** object is created, but since the entire game runs within the constructor their state cannot be read until the game has already finished.

The same mostly applies to steps 6-13, even though steps 7 and 13 utilize a separate

method **play()**, this method is very tightly coupled to the **apples2apples** class, resulting in the same problem of needing to play through an entire game in the constructor before the object is reachable.

While step 14 is difficult to test, as the score allocations happen within the constructor, step 15 is testable through asserting that the highest score after the object is instantiated (and the game thus complete) is correct in relation to the amount of players in the game. As the code is right now, the winning score is seemingly hard coded to be 4, meaning that such an assertion should only be correct for eight or more players.

```
game = new apples2apples( playercount )
// play through the game
maxScore = 0
for i=0 ; i < apples2apples.players.size() ; i++
    if players.get(i).greenApples.size() > maxScore
        maxScore = players.get(i).greenApples.size()
// set desired score to expected value for player count
assert (maxScore == desiredScore)
```

2. Quality attributes, requirements and testability

Why are requirements 16-17 on the previous page poorly written (hint: see the title of this question)? Motivate your answer and what consequences these poorly written requirements will have on development.

The requirements are very open-ended, and should instead focus on the desired structure of the code itself rather than “should be modifiable”. Vague directions such as these might lead developers to spend time trying to fulfill the requirements on sections of code that do not necessarily warrant that level of attention, instead of focusing on the parts where extensibility matters the most.

3. Software Architecture and code review (3p, max 1 page)

Reflect on and explain why the current code and design is bad from:

- an Extensibility quality attribute standpoint
- a Modifiability quality attribute standpoint
- a Testability quality attribute standpoint

Use terminologies from quality attributes: coupling, cohesion, sufficiency, completeness, primitiveness - <https://atomicobject.com/resources/oo-programming/oo-quality>).

As the code is right now, the client and the server use the same **apples2apples** class, but with different constructors. The class itself has no methods associated with it (apart from main??) and instead runs the entire gameplay loop within the constructor itself.

This structure is bad from both an *extensibility* and a *modifiability* standpoint, as extending the functionality of the program necessarily requires a developer to directly modify the existing code. Since most of the functionality in this program is nested within a single class,

the coupling is mostly nonexistent (except for the very tightly coupled **play()** method, that references an **apples2apples** object).

The code is very poor from a cohesion standpoint, since just about everything is handled in the **apples2apples** class instead of being delegated to sub-classes with smaller and more specific tasks. This structure results in a program where *testability* suffers, since there is no good way of testing the constituent parts of the program themselves.

4. Software Architecture design and refactoring

Player-Classes

Instead of one single **player** class with flags defining behaviour, as implemented in the supplied code, I have opted to create an abstract **player** class containing some shared logic as well as several methods that are overridden where behaviour needs to differ between the implementations.

In total there are three classes that currently implement **playerBot**, **playerLocal**, and **playerOnline**. The **first** of these is responsible for any actions a bot needs to take, always defaulting to playing and voting for the card in index zero.

The **second** is responsible for the Local player, i.e.: the player on the same machine as the server. This class handles the I/O part of the game, prompting the player to take game actions. An instance of this class also runs on each online client, as they need to utilize the same I/O functionality.

The **third** class is **playerOnline**, which acts as the server's representation of online players. This class is responsible for sending and receiving data to and from its associated client. Data is packaged in a format of instruction|data|data|data, where the amount of data will depend on the specific instruction.

Associated with the **playerOnline** class we have **onlineManager**, where online players are created and added to the list of players. Whether or not this class is truly necessary can be argued, but I felt that it was reasonable to split this into its own class as we can thus guarantee that the socket used for online communication remains within scope throughout the game.

The reasoning behind this structure is that it allows other methods and classes to effectively ignore the exact composition of players. Regardless if the game is set up with seven bots or only online players, other classes can simply iterate through a list of **player** objects, and since they all implement methods present in the **player** class, they can be invoked regardless of the underlying functionality.

Data classes

There are three main data classes in the current iteration of the game: **playedApple**, **gameStateObject**, and **cardDeck**.

The first, **playedApple** is the way a played card is associated to the player who played it, and only contains fields for the text content of the apple as well as the playerId of its player.

The second, **gameStateObject**, is utilized to create a sharable state that is mutated by the various phases, it contains both decks, as well as data such as the current green apple, the playerId of the current judge, the list of cards played this round, which apple won this round, as well a flag indicating whether or not someone has won yet.

The main reason this implementation was chosen is that it facilitates passing of data between the separate phases of the game (described below) while allowing each **gamePhase** to execute its logic within a void function.

The third is **cardDeck**, which is where the list of cards is read from a .txt file. This class has methods for shuffling the deck, reshuffling the deck, as well as drawing a card. In this implementation of Apples2Apples two card decks are used, one for red apples and one for green apples.

GamePhase-classes

In order to break the game up into more distinct steps an abstract **gamePhase** class was implemented. While this class only has a single method, **execute(data)**, which is blank in the abstract class, it allows us to write implementations of the class that can all be placed into a single list and iterated through to execute the main game loop. This means that when a developer wants to add new game phases, the only place in existing code they'd have to alter is where the **gamePhase**s are inserted into a list, as the **iterate(data)** function in **gamePhaseManager** moves through the list and runs the **execute(data)** function of each object in order.

gamePhaseManager contains a single method: **iterate(data)**, which moves on to the next item in the list of **gamePhase**s, invokes **execute**, and advances the currentPhase variable as to execute the next item in the list when invoked, looping around to the beginning of the list once it's executed the final item.

gamePhase_roundstart is the phase where a new green apple is drawn and presented, as well as a new judge is selected. Judges are selected through incrementing a judgeID variable, resetting it to zero once it's exceeded the total number of players.

gamePhase_play is where non-judges are prompted to play a card for the round. This is done through iterating through the list of players, calling the **play** method on each player. The players each add a **playedApple** to a list, which is then shuffled and ready to use in the judging step.

gamePhase_judge uses the list of played cards that was generated in the play phase, presenting them to all players and invoking the **judge** method on the player who is currently the judge, prompting them to select an apple as the round's winning apple.

gamePhase_roundend is currently the final phase implemented. This phase is responsible for refilling the hand of each player up to seven cards, as well as checking if any player has won yet.

GameEngine-classes

There are two separate game engines, one for the server and one for clients, as their functionality needs to be slightly different.

The first engine class is **gameEngine_host** which is the class used while running the game as a server. This class takes responsibility for tracking the entire state of the game as well as communicating with each connected client. The server is where game phases are utilized, looping through them as previously described to advance the state of the game.

The second engine class is **gameEngine_client**, which is used by each player besides the one playing on the host instance. This client only tracks the player local to it, and acts based on messages received from the host's **playerOnline** class instead of utilizing game phases.

Quality Attribute requirements

In order to satisfy the quality attribute requirements as described in points 16 and 17 (*modifiability*, *extensibility*, and *testability*) the program was broken up into smaller classes, where each class had a much more defined purpose compared to the previous monolithic structure. The implementation of phases and player classes as extensions of an abstract class means that new types of players and phases can be added without impacting existing code at all, improving *extensibility*.

Classes being more compact and having their logic broken up into smaller methods also means that *modifiability* is improved, as a change can happen entirely within one method, and as long as the change is correctly made, it will go unnoticed by the surrounding code.

Testability also becomes greater in my design, as phases can be tested by polling the **gameStateObject** and **player** objects before and after each phase to verify that the state of the game has changed in the way that is expected. Beyond this, methods such as the ones within the player classes can also easily be tested, as they can be executed and then checked without needing to be within the scope of a full game.

Design Patterns

While it is not necessarily an exact implementation, the arrangement of abstract classes extended to have shared methods that can be invoked to execute different logic without knowing the exact implementation can be compared to a **strategy pattern**. This pattern is used both in the **gamePhaseManager** and in the list that manages all the players.

Since the **execute(data)** method in the abstract **gamePhase** class ended up not containing any "default" logic, it could be argued that an interface would have been more appropriate for the purpose, but an abstract class was used to match with the way **player** was implemented, where the default logic inherited from the abstract class was beneficial.

The main benefit of these solutions is that new **player** and **gamePhase** implementations can be written and integrated seamlessly, since they can just be placed into their respective lists and treated as any existing implementation.